

Concurrency Tax from System Structure

A Graph Theory of Processing Overhead

Registry: [\[HOWL-MATH-18-2026\]](#)

Series Path: [\[HOWL-INFO-11-2026\]](#) → [\[HOWL-INFO-12-2026\]](#) → [\[HOWL-INFO-13-2026\]](#) → [\[HOWL-MATH-14-2026\]](#) → [\[HOWL-MATH-15-2026\]](#) → [\[HOWL-MATH-16-2026\]](#) → [\[HOWL-MATH-17-2026\]](#) → [\[HOWL-MATH-18-2026\]](#)

Date: June 2026

DOI: 10.5281/zenodo.20630300

Domain: Information Processing Theory / Applied Mathematics

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.6.

1. The Phenomenon

A function that takes 4 milliseconds in a benchmark takes 23 milliseconds in production. A surgeon who ties a knot in 8 seconds in the simulation lab takes 14 seconds in the operating room. A developer who fixes a bug in 20 minutes when working alone takes 55 minutes during a regular workday with messages, meetings, code reviews, and context switches. A single-threaded program that completes in 100 milliseconds takes 340 milliseconds when running alongside three other processes on the same machine.

Nothing is wrong with the processor in any of these cases. The function is the same function. The knot is the same knot. The bug is the same bug. The program is the same program. The processor's capability has not degraded. What changed is the environment — other activities are competing for shared resources. The work itself is identical. The overhead is the difference, and the overhead is large.

This overhead has names in every domain. In computing it is called contention. In organizations it is called coordination cost. In cognitive science it is called task-switching cost. In manufacturing it is called changeover time. Each domain has its own vocabulary and its own local models for why things take longer than they should. None connects to the others. The computing models use queueing theory. The cognitive models use attentional resource theory. The organizational models use communication complexity. Each explains its own domain and is silent on the rest.

This paper formalizes the overhead as a single computable quantity derived from system structure — the topology of shared resources and competing activities — applicable across all domains. The tool is a graph. The result is an equation.

The vocabulary is small and builds in order. Processing is what any system does when it must act on information — a CPU executing instructions, a surgeon operating, a pilot

flying, a developer debugging. The unit of processing is the **op**: one irreducible transformation by one processor. Processing entropy is the op count a specific processor requires for a specific task. The processor's total capacity is bounded by one inequality: total ops multiplied by average op duration must not exceed the available time budget.

Two counting regimes apply to any processing measurement. The **isolated regime** measures one task on one pipeline with no concurrency — nothing else running, no competition, no interruption. This is where the theoretical minimum cost lives: the ops required for the task itself and nothing else. The **in situ regime** measures the same task in its real execution environment — other tasks competing for the same pipeline, shared resources occupied by others, and the full weight of the concurrent world pressing on the processor.

The difference between in situ cost and isolated cost is the **concurrency tax**: additional ops, inflated op durations, and dead time generated by the execution environment rather than by the task itself. This paper shows that the concurrency tax is derivable from the structure of the system — from which resources are shared and which activities compete for them — rather than being an empirical quantity that can only be measured after the fact.

2. The Five Components

The concurrency tax is not one thing. It is five distinct phenomena that sum to the total overhead. Each has a different mechanism, a different scaling behavior, and a different intervention strategy. Treating them as one undifferentiated “overhead” obscures the structure that makes the tax computable.

Contention. A processing stream needs a resource that another stream currently holds. The needing stream waits. The wait consumes time budget without producing work. The op itself is unchanged — same transformation, same logical content — but its duration inflates by the waiting time. A database query blocked on a connection pool slot occupied by another query still executes the same query when the slot opens. The query cost hasn't changed. The calendar time has. A surgeon waiting for an instrument currently in the assistant's hand still ties the same knot when the instrument arrives. The knot cost hasn't changed. The operating time has. Contention inflates op duration without increasing op count.

Cascade. Activity by one stream changes the context experienced by another stream, invalidating dissolved processing and promoting it back to active cost. Prior work in this series established that dissolved tasks have validity envelopes — regions of context within which dissolution holds. When a competing stream's activity pushes context outside those envelopes, dissolved tasks promote back to active processing, adding ops that would not exist in isolation. A context switch on a CPU evicts cache entries: each subsequent access that would have been a 4-cycle dissolved lookup becomes a 200-cycle active memory fetch. An interruption in the operating room breaks the surgeon's dissolved instrument-handling rhythm, requiring conscious re-engagement with a task that was costing zero. A Slack notification ejects the developer's dissolved codebase navigation from working

memory, requiring reconstruction. Cascade adds ops to a stream that the stream would not have generated on its own.

Coordination. Ops spent managing the concurrency itself — not advancing any stream but maintaining the conditions under which concurrent streams can coexist. Lock acquisition and release. Team communication protocols. Handoff confirmations. Status updates. Merge conflict resolution. These ops exist only because concurrency exists. In the isolated regime, there is nothing to coordinate. Every coordination op is pure overhead — it advances no task, produces no output, and exists solely to manage the interaction between streams that are themselves trying to produce output.

Blocking. Duration where the pipeline is idle because it cannot proceed until a resource held by another stream becomes available. Unlike contention (where the op duration inflates but the pipeline is occupied by the waiting op), blocking is dead pipeline — no op is executing, no transformation is occurring, and time budget is consumed at zero work output. A thread sleeping on a lock is blocking. A surgeon paused while waiting for a frozen section result is blocking. A developer waiting for CI pipeline completion is blocking. The distinction from contention matters because contention inflates an op's measured duration while blocking inserts dead time between ops. Both consume time budget, but they respond to different interventions.

Interleave. Ops spent deciding which stream to service when multiple streams are ready for attention. The meta-decision about what to work on next. When only one stream exists, there is no decision — the pipeline processes whatever is next in that stream. When multiple streams compete, the pipeline must choose: which thread to schedule, which request to handle, whether to respond to the message or continue the current task. The choice itself is an op (or multiple ops for complex priority calculations), and it advances no stream. The scheduler's algorithm for choosing the next process. The surgeon's judgment about whether to address the nurse's question or finish the current suture. The developer triaging between the Slack message, the code review request, and the bug they were investigating. Each interleave decision consumes pipeline capacity that could have been spent on actual work.

The total concurrency tax for a stream is the sum:

tax = contention + cascade + coordination + blocking + interleave

In the isolated regime, all five terms are zero — there are no other streams, no shared resources, no context disruption, nothing to coordinate, nothing to choose between. In situ, each contributes some positive amount. The question is whether those amounts are computable from the system's structure before the system runs.

3. The Contention Graph

The system's structure — which resources are shared and which streams compete for them — is a graph. Making the graph explicit makes the concurrency tax derivable.

Define the **contention graph** $G = (R, S, E)$ with three components.

$\mathbf{R} = \{r_1, r_2, \dots, r_m\}$ is the set of shared resources. A shared resource is anything that serves one stream at a time or degrades when serving multiple streams simultaneously. In computation: a CPU core, a cache line, a memory bus, a database connection pool slot, a disk I/O channel, a network socket. In surgery: a specific instrument, the surgeon’s visual focus, the circulating nurse’s hands, the anesthesia monitor’s attention, the operative field itself. In an office: a meeting room, the CI pipeline, the team lead’s decision-making capacity, a shared document under concurrent editing, the deployment environment.

$\mathbf{S} = \{s_1, s_2, \dots, s_n\}$ is the set of processing streams. A stream is a sequence of ops that constitutes one logical task being executed concurrently with others. In computation: a thread, a process, a request being handled. In surgery: the primary procedure, patient monitoring, instrument management, anesthesia management, documentation. In an office: each developer’s current task, each ongoing code review, each active conversation thread.

$\mathbf{E} \subseteq \mathbf{S} \times \mathbf{R}$ is the set of edges connecting streams to resources. An edge (s_i, r_j) means stream s_i requires resource r_j at some point during its execution. Each edge carries three annotations:

Hold duration $h(s_i, r_j)$: how long stream s_i holds resource r_j per access. A database query holds a connection for the query duration. A surgeon holds an instrument for the duration of its use. A developer holds the CI pipeline for the duration of a build.

Access frequency $f(s_i, r_j)$: how often stream s_i requires resource r_j per unit time. A process accesses L1 cache thousands of times per millisecond. A surgeon reaches for the primary instrument tens of times per minute. A developer pushes to CI a few times per day.

Criticality $c(s_i, r_j)$: whether stream s_i blocks completely when resource r_j is unavailable (critical = 1) or can proceed with degraded performance or alternative resources (critical = 0, or a continuous value between). A thread requiring a mutex cannot proceed without it — critical. A developer waiting for a code review can switch to another task — non-critical but with switching cost.

The contention graph makes competition structure visible. Two streams that share no resource neighbors cannot contend — they run as if isolated regardless of how many other streams exist. Two streams connected to many common resources contend frequently. A resource connected to many streams is a bottleneck — it mediates contention among all its connected streams. A stream connected to many resources is a high-exposure participant — it has many channels through which other streams can affect it.

The **resource contention degree** of resource r_j is the number of streams connected to it: $\deg(r_j) = |\{s_i : (s_i, r_j) \in \mathbf{E}\}|$. A resource with degree 1 is private — no contention possible. A resource with degree n (connected to every stream) is maximally shared.

The **stream exposure** of stream s_i is the number of resources it requires: $\exp(s_i) = |\{r_j : (s_i, r_j) \in \mathbf{E}\}|$. A stream with exposure 1 uses one shared resource — its contention surface is minimal. A stream with high exposure uses many shared resources — it has many points at which other streams can interfere.

The **contention overlap** between streams s_i and s_j is the number of resources they share: $\text{overlap}(s_i, s_j) = |\{r_k : (s_i, r_k) \in E \wedge (s_j, r_k) \in E\}|$. Zero overlap means the streams cannot directly contend. High overlap means frequent potential for interference.

4. Deriving Tax Components from Graph Properties

Each component of the concurrency tax is a function of the contention graph's structure and annotations. This section derives each one.

Contention cost. For resource r_j with degree k (shared by k streams), the probability that a stream finds it occupied depends on the aggregate demand. Each connected stream s_i generates demand at rate $f(s_i, r_j)$ with hold duration $h(s_i, r_j)$. The utilization of resource r_j is:

$$\rho(r_j) = \sum_i f(s_i, r_j) \times h(s_i, r_j)$$

summed over all streams connected to r_j . When $\rho < 1$, the resource is below saturation and contention is probabilistic — sometimes available, sometimes occupied. When $\rho \geq 1$, the resource is saturated and contention is guaranteed — streams queue.

For the subsaturation regime, model resource r_j as a single server with aggregate arrival rate $\lambda_j = \sum_i f(s_i, r_j)$ and mean service time $\mu_j^{-1} = \text{mean of } h(s_i, r_j) \text{ across connected streams}$. The expected waiting time per access, from standard queueing results, is:

$$W(r_j) = \rho(r_j) / (\mu_j \times (1 - \rho(r_j)))$$

This is the M/M/1 waiting time, applicable when arrivals are approximately Poisson and service times approximately exponential. For more structured access patterns, the formula adjusts (M/D/1 for deterministic service, M/G/1 for general service distributions), but the functional dependence on ρ remains: waiting time increases with utilization and diverges as ρ approaches 1.

The total contention cost for stream s_i is the sum of expected waiting time across all resources it uses:

$$\text{contention}(s_i, G) = \sum_j f(s_i, r_j) \times W(r_j)$$

Each access to each shared resource contributes its frequency-weighted expected wait. Resources with high utilization contribute disproportionately. Resources with low utilization contribute little.

Cascade cost. Stream s_i has a dissolution inventory — a set of dissolved elements with validity envelopes in context space, as developed in prior work. Other streams' activities on shared resources constitute context changes that may violate those envelopes.

Define the **cascade coupling coefficient** $\kappa(s_i, s_j, r_k)$ as the expected number of cascade promotions in stream s_i per unit of activity by stream s_j on shared resource r_k . This coefficient depends on how much of s_j 's activity on r_k falls outside s_i 's validity envelopes for elements whose dissolution involves r_k . A context switch on a CPU where two processes share L1 cache has a high κ because the incoming process's memory accesses evict the

outgoing process's cached data. Two processes sharing a database connection pool but accessing different tables have low κ because holding a connection doesn't change the other process's processing context.

The total cascade cost for stream s_i is:

$$\text{cascade}(s_i, G) = \sum_j \neq_i \sum_k \kappa(s_i, s_j, r_k) \times \text{activity}(s_j, r_k) \times \text{recovery_cost}(s_i)$$

where $\text{activity}(s_j, r_k)$ is stream s_j 's usage rate of shared resource r_k , and $\text{recovery_cost}(s_i)$ is the average op cost to restabilize a promoted element in stream s_i . The triple sum reflects that cascade damage to stream s_i comes from every other stream through every shared resource, weighted by the coupling coefficient.

Coordination cost. Each shared resource may require coordination ops to manage concurrent access. Define $\text{coord}(r_j)$ as the coordination cost per access to resource r_j — the ops required for acquisition protocol, release protocol, and state consistency verification. For a mutex: the lock and unlock operations. For a surgical instrument: the request, confirmation, and handoff. For a code review: the request, review cycle, and merge protocol.

The total coordination cost for stream s_i is:

$$\text{coordination}(s_i, G) = \sum_j f(s_i, r_j) \times \text{coord}(r_j)$$

Every access to every shared resource incurs its coordination cost. Resources with expensive protocols contribute more. Resources accessed frequently contribute more.

Blocking cost. When resource r_j is critical for stream s_i (criticality $c(s_i, r_j) = 1$) and r_j is occupied, the stream blocks — pipeline is idle, time budget consumed, zero ops executed. The blocking cost is the expected time spent blocked:

$$\text{blocking}(s_i, G) = \sum_j f(s_i, r_j) \times c(s_i, r_j) \times W(r_j)$$

This is similar to contention cost but applies only to critical-path resources (weighted by criticality) and represents dead time rather than inflated op duration. For non-critical resources ($c = 0$), the stream can switch to another activity during the wait — the cost shifts from blocking to interleave.

Interleave cost. When the pipeline must choose between ready streams, the choice consumes ops. Define $\text{interleave_cost}(k)$ as the op cost of choosing between k ready streams. For a simple round-robin scheduler: $\text{interleave_cost} \approx \text{constant}$ (one op regardless of k). For a priority-based scheduler: $\text{interleave_cost} = O(\log k)$ (priority queue operations). For a complex multi-criteria decision (the surgeon evaluating whether to respond to the nurse): interleave_cost may be several ops involving evaluation of urgency, current task interruptibility, and consequence assessment.

The total interleave cost for stream s_i depends on how often it is preempted or must choose, which depends on the arrival rate of competing demands:

$$\text{interleave}(s_i, G) = \text{preemption_rate}(s_i, G) \times \text{interleave_cost}(\text{active_streams}(G))$$

where preemption_rate is how often stream s_i is interrupted by a competing stream, and active_streams is the number of concurrent streams ready for the pipeline.

5. The Concurrency Tax Equation

The total concurrency tax for stream s_i in contention graph G assembles all five components:

$$\text{tax}(s_i, G) = \text{contention}(s_i, G) + \text{cascade}(s_i, G) + \text{coordination}(s_i, G) + \text{blocking}(s_i, G) + \text{interleave}(s_i, G)$$

The in situ cost of stream s_i is:

$$\text{cost_insitu}(s_i) = \text{cost_isolated}(s_i) + \text{tax}(s_i, G)$$

The total system cost across all streams is:

$$\text{cost_total}(G) = \sum_i \text{cost_isolated}(s_i) + \sum_i \text{tax}(s_i, G)$$

The tax has several structural properties that follow from the derivations.

The tax is a system property, not a stream property. Each stream's tax depends on every other stream through shared resources. Adding a stream increases every other stream's tax: more contention on shared resources, more cascade risk from context changes, more coordination per access, more blocking from occupied resources, more interleave decisions when the pipeline chooses. The tax is computed from the graph, not from any stream in isolation.

The marginal tax of the (k+1)th stream is generally superlinear. Each additional stream adds tax to every existing stream. The k+1th stream contends with k existing streams on shared resources, generates cascade coupling with k streams, adds coordination ops for k potential interactions, adds blocking probability for k competitors, and adds one more option to every interleave decision. The total tax increase from adding one stream grows with the number of existing streams.

The tax dominates at high concurrency. When the number of streams is small relative to the number of resources, tax is small — most resources are uncontested, cascades are rare, coordination is light. As stream count increases, utilization rises, contention increases nonlinearly (the queueing wait time diverges as $\rho \rightarrow 1$), cascades become frequent, coordination accumulates, blocking becomes common, and interleave decisions become continuous. At some stream count, the tax per stream exceeds the stream's isolated cost — concurrency overhead exceeds useful work. Beyond this point, adding streams decreases total throughput.

The tax structure determines the intervention. Contention-dominated tax responds to resource replication. Cascade-dominated tax responds to isolation and wider dissolution envelopes. Coordination-dominated tax responds to protocol simplification and dissolution of coordination routines. Blocking-dominated tax responds to asynchronous design. Interleave-dominated tax responds to scheduling policy. Diagnosing which component dominates — which requires the component decomposition, not just the total — determines which intervention works.

6. Architectural Motifs

Certain contention graph topologies recur across domains. Each has a characteristic tax profile derivable from its structure.

Star topology. One central resource connected to all streams. All contention concentrates on one point. The central resource has degree n (all streams), utilization $\rho = n \times \bar{f} \times \bar{h}$, and waiting time that diverges as $\rho \rightarrow 1$. Peripheral resources (if any) are private — no contention.

Tax is dominated by contention and blocking on the central resource. As stream count increases, the central resource saturates. The tax scaling is steep: $O(n/(1-n\bar{f}\bar{h}))$ for contention, approaching divergence as $n\bar{f}\bar{h} \rightarrow 1$.

Examples: single database connection pool serving all application threads. Single CI pipeline shared by all developers. Single meeting room for all teams. Surgeon’s visual attention as sole shared resource during complex multi-participant procedures. In each case, the central resource is the system’s throughput ceiling.

Mitigation: replicate the central resource. Multiple connection pools. Parallel CI runners. Multiple meeting rooms. A second surgeon or an assistant with independent visual field. Replication transforms the star into a partitioned or hierarchical topology.

Chain topology. Resources shared pairwise along a sequence. Stream 1 shares resource A with stream 2. Stream 2 shares resource B with stream 3. Stream 3 shares resource C with stream 4. No stream shares a resource with a non-adjacent stream.

Tax propagates along the chain. A delay in stream 1 holds resource A, blocking stream 2, which holds resource B, blocking stream 3. The propagation delay through a chain of length L accumulates: total propagation time is the sum of individual contention delays, which can amplify small initial delays into large end-to-end latencies.

Cascade coupling is nearest-neighbor only — a stream can only be disrupted by the streams it shares resources with. Coordination is pairwise. Blocking propagates sequentially.

Examples: pipeline architectures where each stage hands output to the next through a shared buffer. Assembly lines where stations share tooling with adjacent stations. Sequential approval chains where each approver’s output is the next’s input.

Mitigation: buffer between chain links (decouples propagation), parallel chains (redundancy), or asynchronous handoff (converts blocking to non-blocking).

Complete graph (all-to-all). Every stream shares at least one resource with every other stream. Maximum contention topology.

Tax scales quadratically. Every pair of streams can contend on shared resources. Adding the $(k+1)$ th stream adds k new contention relationships. Total coordination overhead is

$O(n^2)$ because every pair may need to coordinate. Cascade exposure is maximal — every stream's activity can disrupt every other stream.

Examples: shared-everything symmetric multiprocessor architectures. Open-plan offices where every person can see and interrupt every other person. Team meetings where every participant speaks to every other. Shared mutable state accessed by all threads.

This topology produces the maximum possible concurrency tax for a given stream count. The tax dominates quickly — useful throughput peaks at a low stream count and degrades beyond it.

Mitigation: partition. Divide streams into groups that share resources within groups but not between. Transform the complete graph into a partitioned topology.

Partitioned topology. Streams divided into groups of size k . Resources shared within groups but not between. No inter-group edges.

Tax is bounded by within-group contention. Each group experiences tax as if it were a system of k streams. Groups are independent — adding streams to one group does not affect another. Total tax scales with group size k , not total stream count n .

Examples: NUMA domains where each processor group has its own memory controller. Pod-based team structures where each pod has its own resources and communicates with other pods only through defined interfaces. Microservice architectures where each service manages its own state.

This topology achieves linear scaling with total streams as long as group size is bounded. The cost is that cross-group work requires inter-group coordination (a new type of coordination cost at a higher level), but within-group work is insulated from the rest of the system.

Hierarchical topology. Resources organized in tiers. Local resources (private or small-group) are cheap to access and rarely contested. Global resources (system-wide) are expensive and highly contested. Streams access local resources frequently and global resources rarely.

Tax has two components: low local tax (frequent access, low contention) and high global tax (rare access, high contention). The effective tax is dominated by the access frequency distribution — if most accesses are local, the tax is low despite high global contention.

Examples: CPU cache hierarchies. L1 private (zero contention), L2 shared by core pair (low contention), L3 shared by all cores (moderate contention), main memory (high contention). Organizational hierarchies: team-level decisions (fast, uncontested), department-level decisions (slower, some contention), company-level decisions (slow, high contention). The hierarchy's depth and branching factor determine the tax profile.

Tax scales as $O(\log n)$ for the common case (most accesses resolved locally) with $O(n)$ penalties for global access (resolved at the root). The tradeoff is latency versus sharing — local resolution is fast but isolated, global resolution is slow but coordinated.

7. Human and Organizational Processors

The contention graph formalism extends to human processors without modification. The shared resources are cognitive and organizational rather than computational. The mechanisms are the same.

The human processor has one pipeline: conscious attention. It processes one op at a time. Multiple concurrent demands compete for this pipeline. The demands are the streams. The shared resources they compete for include:

Working memory. Limited capacity, shared across all active tasks. A developer holding a complex data flow in working memory uses most of the available capacity. An interruption that requires understanding a different context competes for the same memory. Working memory is a high-contention resource with high cascade coupling — evicting one task's context to load another's invalidates the first task's dissolved processing chains.

Visual attention. One focal point at a time. The surgeon's eyes can attend to one location. A notification on screen competes with the code being read. A new instrument in the operative field competes with the tissue being dissected.

Motor system. One pair of hands doing one thing. The developer's hands can type one thing. The surgeon's hands can manipulate one instrument. Motor contention is rare for cognitive work (the bottleneck is usually attention, not hands) but common for physical work.

Communication bandwidth. One conversation at a time at full attention. The developer can engage with one Slack thread, one meeting, or one code review conversation at full quality. Concurrent conversations degrade all of them.

The contention graph for a developer during a workday maps directly. Streams: primary development task, code review requests, Slack conversations, email, meetings, incident monitoring, administrative tasks. Resources: working memory, visual attention, communication bandwidth, CI pipeline, shared codebase (merge conflicts), team lead's decision capacity. Edges connect each stream to the resources it requires, annotated with hold durations, access frequencies, and criticality.

The tax derivation follows the same equations. Contention cost: how often each resource is occupied by a competing stream when the developer needs it. Cascade cost: how much each interruption disrupts dissolved processing. Coordination cost: ops spent on protocols (meeting agendas, status updates, handoff procedures). Blocking cost: time spent waiting for resources (CI pipeline, code review, decisions). Interleave cost: ops spent deciding which stream to attend to.

The cascade component deserves particular attention for human processors because it is typically the dominant term. When a developer is interrupted, the interruption itself may take two minutes. But the cascade — the dissolution promotions caused by working memory eviction — adds 15-25 minutes of recovery time as the developer reconstructs their mental model of the code. The interruption's direct cost (2 minutes of coordination) is small. The cascade cost (15-25 minutes of re-orientation) dominates by an order of magnitude.

The contention graph explains this quantitatively. The primary development task holds working memory as a critical resource with long hold duration (the entire development session). Working memory has a small capacity and a high cascade coupling coefficient — evicting its contents promotes many dissolved processing chains simultaneously. An interruption that requires working memory (understanding a different context) forces eviction. The cascade severity is the count of promoted elements, which for deep programming work can be high — the developer has dissolved dozens of code navigation patterns, naming conventions, data flow paths, and state assumptions that all depend on the working memory context being maintained.

The graph also predicts which interruptions are expensive and which are cheap. An interruption that does not require working memory (a timer going off, a brief physical movement) has low cascade coupling — it doesn't evict working memory contents and therefore doesn't trigger a cascade. An interruption that requires loading a different complex context (switching to a different project's codebase) has maximum cascade coupling — it evicts everything. The concurrency tax of an interruption is predictable from the edge between the interrupting stream and the working memory resource, specifically from the cascade coupling coefficient on that edge.

For organizational processors, the graph extends one level further. The shared resources are organizational structures: the deployment pipeline, the release schedule, the product roadmap, the team lead's attention. Streams are the concurrent projects or work items the organization is pursuing. The contention graph at the organizational level has the same structure — resources, streams, edges with annotations — and the same tax components apply.

The organizational cascade component manifests as organizational disruption. A re-organization (context change) that invalidates team members' dissolved coordination patterns — who to ask for what, how decisions get made, where to find information — generates cascade promotions across the organization. Each team member must reconstruct their organizational navigation from active processing. The cascade severity is the count of organizational patterns that promoted across all affected team members, which for a large re-organization can be enormous.

8. The Expert Tax Discount

Expertise reduces the concurrency tax. Not because the expert's environment is simpler — the contention graph is the same — but because three of the five tax components are themselves processing operations that dissolve through repetition.

Interleave dissolution. The decision about which stream to service is itself an op. When it is performed repeatedly in consistent conditions, it dissolves like any other op. The experienced surgeon's decision about whether to address the nurse's question or finish the suture is dissolved — the decision fires structurally based on the urgency pattern and the task's interruptibility, without conscious pipeline allocation. The novice surgeon must consciously evaluate each interruption: Is this urgent? Can my current step be paused?

What are the consequences of each choice? Three to five ops per interleave decision for the novice versus zero for the expert.

Cascade resistance. Prior work established that wider validity envelopes reduce cascade severity. Experts have wider envelopes — their skills dissolved under varied conditions, so context changes stay within envelopes rather than crossing them. The expert developer’s codebase navigation survives a brief Slack interruption because the navigation dissolved under conditions that included occasional interruptions. The novice developer’s fragile, narrowly-dissolved navigation breaks on the same interruption.

The cascade coupling coefficient κ is lower for the expert on every shared resource edge. The same interruption that promotes five working memory items for the novice promotes one (or none) for the expert. The cascade component of the tax shrinks without any change to the contention graph — the graph is the same, but the expert’s dissolution inventory has wider envelopes.

Coordination dissolution. The surgical team that has worked together for years coordinates with fewer ops than a newly assembled team. A glance replaces a sentence. A gesture replaces a verbal instruction. The handoff protocol for passing an instrument has dissolved from an explicit request–confirmation–transfer sequence (five ops) to an implicit anticipation–extend–accept sequence (one op or zero if fully dissolved). The coordination cost per access to the shared resource decreases through dissolution of the coordination protocol itself.

Contention cost and blocking cost do not dissolve meaningfully. If the resource is occupied, the expert waits the same duration as the novice — waiting is not a skill that improves with practice. The expert may use the wait time more productively (by working on a non-blocked stream without cascade cost, which is itself an interleave dissolution), but the raw wait time is structural.

Define the **tax discount** as the ratio of expert to novice concurrency tax for the same contention graph:

$$\delta(\text{expert, novice, } G) = \text{tax}(\text{expert, } G) / \text{tax}(\text{novice, } G)$$

This ratio is less than 1. The discount decomposes by component:

$\delta_{\text{interleave}}$: typically large (experts have dissolved interleave decisions)

δ_{cascade} : typically large (experts have wider validity envelopes)

$\delta_{\text{coordination}}$: moderate (dissolves with team experience, not just individual)

$\delta_{\text{contention}}$: near 1 (waiting time is structural)

δ_{blocking} : near 1 (blocking time is structural)

The overall discount is dominated by cascade and interleave in environments with frequent context changes (like the operating room or the developer’s workday), and dominated by coordination in environments with high coordination protocols (like surgical teams or military units).

The discount is itself a property of the contention graph — some graph topologies offer more room for discount than others. A graph with few shared resources has small tax and therefore small discount (there is little overhead to dissolve). A graph with many shared resources and high cascade coupling has large tax and large potential discount. The maximum discount occurs in high-concurrency environments with experienced processors: precisely the conditions where the tax would be crushing for novices.

9. Architectural Interventions as Graph Transformations

Every architectural intervention that reduces overhead corresponds to a transformation of the contention graph. Making the transformation explicit makes the expected tax reduction computable before implementation.

Resource replication (reduces contention and blocking). Duplicating a shared resource splits one high-degree node into two lower-degree nodes. If resource r_j with degree n is replicated into r_j' and r_j'' with degrees $n/2$ each, the utilization of each drops: $\rho_{\text{new}} = \rho_{\text{old}} / 2$. The waiting time drops more than proportionally because the queueing wait time is convex in utilization — halving utilization more than halves waiting time. The total contention and blocking reduction is computable from the degree reduction and the utilization formula.

Example: replacing a single database connection pool (degree 100, utilization 0.85, wait time 5.7 time units per access) with two pools (degree 50 each, utilization 0.425 each, wait time 0.74 time units per access). The contention reduction is 87% — not 50% — because the queueing function is nonlinear.

Isolation (reduces cascade). Preventing one stream’s activity from affecting another stream’s context. In the contention graph, isolation removes cascade coupling coefficients on shared resource edges. Full isolation (separate address spaces, separate working environments) sets κ to zero on all edges between isolated streams. Partial isolation (noise-canceling headphones, cache partitioning) reduces κ without eliminating it.

Example: process-level cache partitioning on a multi-core CPU. Assigning each process a dedicated cache partition eliminates cache eviction cascades between processes. The cascade coupling coefficient κ drops to zero on the L1 and L2 cache edges. The cascade component of the concurrency tax for each process drops to zero for cache-mediated disruption. The total tax reduction is computable from the prior cascade cost on those edges.

Protocol simplification (reduces coordination). Reducing the ops required for managing concurrent access to shared resources. In the contention graph, protocol simplification reduces $\text{coord}(r_j)$ — the coordination cost per access — on specific resources.

Example: replacing a three-phase commit protocol (6 ops per coordination point) with an optimistic concurrency protocol (1 op per coordination point, occasional retry). The coordination cost per access drops by 5/6. The total coordination reduction across all

streams accessing that resource is computable from the access frequency and the per-access cost reduction.

Asynchronous decoupling (reduces blocking). Allowing streams to proceed without waiting for resources, deferring the resource need to a later time when it may be available. In the contention graph, asynchronous decoupling changes the criticality annotation on edges from $c = 1$ (blocking) to $c = 0$ (non-blocking). The blocking component drops to zero on those edges. The cost shifts to interleave (the stream must decide what to do while waiting) and possibly coordination (tracking the deferred request), but these are typically cheaper than blocking.

Example: replacing synchronous code review (developer blocks until reviewer responds) with asynchronous review (developer switches to another task, reviews arrive as notifications). The blocking cost on the “reviewer attention” resource drops to zero. The interleave cost increases slightly (developer must manage task switching) and cascade cost may increase (switching tasks may evict working memory), but the total tax typically decreases because blocking time is pure dead time while interleave and cascade at least permit some useful work.

Scheduling policy (reduces interleave). Replacing deliberative interleave decisions with structural (dissolved) scheduling. In the contention graph, scheduling policy reduces `interleave_cost` by replacing per-decision evaluation with a pre-computed rule.

Example: a developer adopting time-blocking — defined two-hour blocks of uninterrupted focus with all notifications deferred to between blocks. The interleave decision within a block is dissolved (the rule is: don’t switch). The interleave cost during blocks drops to zero. Between blocks, the developer processes accumulated interruptions in a batch, which is cheaper than individual interrupt-driven responses because the context switches happen at natural boundaries rather than mid-task.

For each intervention, the expected tax change is:

$$\Delta \text{tax} = \text{tax}(s_i, G_{\text{new}}) - \text{tax}(s_i, G_{\text{old}})$$

where G_{new} is the contention graph after the transformation and G_{old} is the graph before. This is computable from the graph transformation and the tax equation, making the cost-benefit of each intervention evaluable before implementation.

10. Computational Test Case

Ground the formalism in a system where every quantity is measurable to the nanosecond.

Multi-process system on a four-core CPU. Four processes (P1–P4) running simultaneously on four cores sharing a memory hierarchy.

The contention graph has four streams (P1–P4) and five resources: L1 caches (four private — degree 1 each, no contention), L2 caches (two shared between core pairs — degree 2 each), L3 cache (one shared by all four — degree 4), memory bus (one shared by all four — degree 4), and memory controller (one shared by all four — degree 4).

Edge annotations from hardware performance counters during an initial profiling run:

For the L2 cache (shared between P1 and P2): P1 accesses L2 at 50,000 accesses/ms with mean hold time 12 cycles. P2 accesses at 45,000 accesses/ms with mean hold time 15 cycles. Utilization: $(50,000 \times 12 + 45,000 \times 15) / (\text{cycles per ms per L2 bank})$. The contention cost per access is derivable from the utilization.

For the L3 cache (shared by all four): aggregate access rate and per-process hold times from LLC-reference and LLC-miss counters. Utilization and contention wait derivable from aggregate demand.

For the memory bus: bandwidth utilization from memory controller counters. Contention model as a bandwidth-limited resource rather than a discrete server — queueing theory applies with capacity as the service rate.

Cascade measurement. Run each process in isolation and record cache miss rates. Run all four simultaneously and record cache miss rates. The difference — additional cache misses in the concurrent run — is the cascade component. Each additional miss at L2 costs approximately 30 cycles. Each additional miss at L3 costs approximately 200 cycles. The cascade cost in cycles is directly measurable.

Coordination measurement. Atomic operations and lock contention visible through hardware contention counters. Each atomic operation has a base cost (a few cycles) that inflates under contention (potentially hundreds of cycles). The coordination cost is the sum of atomic operation costs minus their uncontended baseline.

Blocking measurement. Time processes spend in I/O wait or lock sleep, visible through scheduler statistics. Direct measurement of dead pipeline time per process.

Interleave measurement. Scheduler overhead measurable through profiling. Time spent in the scheduler's decision code per context switch times number of context switches.

Prediction versus measurement. Compute the predicted tax from the contention graph (structure + annotations) using the equations from Sections 4 and 5. Compare against the measured tax (in situ cost minus isolated cost per process). The prediction should approximate the measurement within a bound determined by the accuracy of the queueing model assumptions and the stability of the workload during measurement.

For a secondary validation, introduce architectural interventions and verify predicted tax changes:

Intervention 1: Pin P1 and P2 to cores sharing one L2, P3 and P4 to the other. Predicted effect: reduced L2 contention for the pairs, unchanged L3 and memory contention. Measure and compare.

Intervention 2: Partition L3 cache giving each process a dedicated quarter. Predicted effect: cascade coupling κ drops to zero on L3 edges; cascade component eliminated for L3-mediated disruption. Contention on L3 also eliminated since partitions are private. Measure and compare.

Intervention 3: Reduce P4's priority so it only runs when no other process is ready. Predicted effect: P1–P3 experience near-zero contention from P4; P4's tax increases (it waits

for all others). Total system tax may decrease if P4's contribution to others' tax was large. Measure and compare.

Each intervention is a specific graph transformation with a predicted tax change from the equation. The measured change validates (or invalidates) the prediction and calibrates the model for this workload class.

11. Organizational Test Case

The contention graph applies equally to human and organizational processors, though measurement is softer. A software development team provides a test case intermediate between precise hardware measurement and purely qualitative organizational observation.

Team of six developers working on a shared codebase with shared infrastructure.

The contention graph has six streams (developer work tasks) and six resources: shared codebase (merge conflicts — degree 6), CI pipeline (build queue — degree 6), team lead attention (decisions and unblocking — degree 6), code review pool (each PR needs a reviewer — degree 6), staging environment (one shared — degree 6), and meeting time (calendar — degree 6).

All resources have degree 6 — this is close to a complete graph topology, which predicts high tax. The annotation data comes from existing team tools:

CI pipeline: build duration from CI logs (hold time h), build frequency from commit history (access frequency f), build queue depth (utilization ρ). If average build takes 12 minutes and the team submits 30 builds per day across a 10-hour workday, utilization is $30 \times 12 / 600 = 0.6$. Expected wait time per build: $\rho / (\mu (1 - \rho)) \approx 0.6 / ((1/12)(0.4)) \approx 18$ minutes per build.

Code review: time-to-first-review from PR timestamps (wait time), review request frequency from PRs opened per day (access frequency), reviewer availability from calendar analysis. If reviews take an average of 2 hours to begin and developers open 2 PRs per day, the blocking cost is approximately 4 hours per developer per day waiting for reviews — a third of the workday.

Team lead attention: decision request rate from Slack/email (access frequency), time per decision from interaction logs (hold time), queue depth when multiple developers need attention simultaneously (contention).

Meeting time: total meeting hours per developer per week from calendar (coordination cost), post-meeting recovery time measured by time to first commit after a meeting (cascade cost).

Predicted tax components. Contention: primarily CI pipeline (18 minutes per build wait at $\rho = 0.6$) and code review (2 hours per PR). Cascade: primarily from meeting context switches and Slack interruptions (estimated 15-25 minutes recovery per interruption, 4-6 interruptions per day = 60-150 minutes of cascade per developer per day). Coordination: meetings plus standup plus async communication (estimated 1.5-2 hours

per day). Blocking: CI wait plus review wait (estimated 2-4 hours per day). Interleave: task-switching decisions (estimated 15-30 minutes per day).

Total predicted tax per developer per day: approximately 4-7 hours. For a 10-hour workday, this leaves 3-6 hours of isolated-equivalent productive work. This matches industry measurements of developer productivity that consistently find 3-5 hours of focused coding per 8-10 hour workday — the remainder is concurrency tax.

Intervention predictions. Adding a second CI runner: transforms star topology on CI resource into two-node partition. Predicted contention reduction on CI: from 18 minutes per build to approximately 4 minutes (utilization halves, wait time drops more than proportionally). Total predicted saving: approximately 30 minutes per developer per day.

Instituting no-meeting mornings: removes meeting stream's edges from the contention graph for half the day. Predicted cascade reduction: half as many meeting-induced cascade events. Predicted saving: approximately 30-60 minutes per developer per day in recovered cascade time.

Switching to async code review with a 4-hour SLA: changes code review from blocking ($c = 1$) to non-blocking ($c = 0$). Predicted blocking reduction: 2 hours per developer per day converted from dead wait to productive work on other tasks. Interleave and cascade costs increase modestly (task switching to fill review wait), but net benefit is positive.

Each prediction is testable against before/after productivity metrics — commit frequency, time to resolution, developer satisfaction surveys, and direct measurement of focused work hours through calendar and tool analysis.

12. Predictions

The formalism produces testable predictions across domains.

Prediction 1: Tax predictability. The concurrency tax for a given stream is predictable from the contention graph with accuracy proportional to the completeness of edge annotations. Systems with measurable access frequencies, hold durations, and cascade coupling coefficients yield tighter predictions. Systems where these quantities are estimated loosely yield proportionally looser predictions.

Prediction 2: Intervention specificity. Architectural interventions that remove edges or reduce edge weights in the contention graph produce measurable reductions in the specific tax components those edges contribute to, and the magnitude is predictable from the graph transformation. An intervention targeting contention reduces contention and does not reduce cascade. An intervention targeting cascade does not reduce contention. The tax equation decomposes the effect.

Prediction 3: Motif-determined scaling. The marginal tax of adding the $(k+1)$ th stream follows the scaling law of the graph motif. Star and complete topologies: superlinear. Chain: linear with propagation. Partitioned: constant (within groups). Hierarchical: logarithmic for common case. The scaling law is determined by topology, not by domain.

Prediction 4: Expert discount components. Expert concurrency tax is measurably lower than novice tax on the same contention graph. The discount is largest for cascade and interleave (which dissolve through experience) and smallest for contention and blocking (which are structural). Measurable by comparing expert and novice cost on the same tasks in the same concurrent environment.

Prediction 5: Brooks’s Law derivation. For all-to-all team topologies, total coordination ops grow as $O(n^2)$ with team size. At the stream count where marginal coordination tax exceeds marginal isolated work capacity, adding team members decreases throughput. The crossover point is computable from the graph — specifically from the coordination cost per edge and the edge count growth rate.

Prediction 6: Optimal concurrency. For any contention graph, there exists an optimal stream count n^* where total throughput (total isolated work minus total tax) is maximized. Below n , *adding streams increases throughput (the new stream’s isolated work exceeds its marginal tax)*. Above n , *adding streams decreases throughput (the marginal tax exceeds the new stream’s isolated work)*. The optimal point is computable from the graph’s scaling law: $n^* = \operatorname{argmax}_n \{ n \times \text{work_isolated} - \text{tax_total}(n, G) \}$.

13. Scope and Open Problems

This paper establishes the contention graph as the structural representation of concurrent system topology, derives the five-component concurrency tax from graph properties, characterizes architectural motifs with their scaling laws, extends the formalism to human and organizational processors, derives the expert tax discount from dissolution of overhead components, and specifies architectural interventions as graph transformations with predictable effects.

The following remain open.

Dynamic contention graphs. The graph is treated as static throughout this paper. In practice, streams start and end, resource requirements change, and the graph evolves during execution. Formalizing time-varying contention graphs — where edges appear and disappear, annotations change, and the tax must be computed as a time-varying function rather than a steady-state quantity — extends the framework to systems with changing workload mixes.

Degrading resources. The formalism assumes resources are either available or occupied. Many real resources degrade continuously under concurrent use rather than blocking: a network link with increasing latency under load, a manager whose decision quality decreases with interruption frequency, a shared codebase where merge conflict probability increases with concurrent edits. Extending the model from binary exclusive access to continuous degradation functions on shared resources would capture these effects.

Nested contention. A developer experiences cognitive contention (internal streams competing for working memory) while simultaneously experiencing organizational contention (competing with teammates for shared infrastructure). The two contention graphs inter-

act — high cognitive tax reduces the developer’s output rate, which increases the organizational blocking time for teammates waiting on their deliverables. Formalizing coupled contention graphs at different levels (cognitive, team, organizational) and their cross-level interactions is an open structural problem.

Component interaction. The five tax components are summed in the current formulation. In practice, they may interact: high contention increases the probability of cascade (longer waits mean more time for context disruption), high cascade increases interleave cost (more promoted elements competing for the pipeline), and high blocking increases cascade exposure (idle pipeline leaves dissolved elements vulnerable to context change). Formalizing these interactions as second-order terms would improve prediction accuracy for high-load systems.

Tax measurement standardization. The paper defines what the concurrency tax consists of and how it derives from graph structure, but does not standardize measurement protocols. Specifying operational procedures for measuring each component in each domain — what instruments to use, what baselines to establish, how to isolate components — would make cross-domain and cross-study comparison rigorous.

Appendix: Supporting Tables

HOWL-MATH-18-2026

Table A: Formal Definitions

Symbol	Name	Definition	Unit	Domain
$G = (R, S, E)$	Contention graph	Graph with shared resources R , processing streams S , and edges E connecting streams to resources they require	—	Universal
$R = \{r_1, \dots, r_m\}$	Resource set	Set of shared resources; each serves one stream at a time or degrades under concurrent use	—	System-specific

Symbol	Name	Definition	Unit	Domain
$S = \{s_1, \dots, s_n\}$	Stream set	Set of concurrent processing streams competing for resources and pipeline	—	System-specific
$E \subseteq S \times R$	Edge set	Connections between streams and the resources they require; each edge annotated with h, f, c	—	System-specific
$h(s_i, r_j)$	Hold duration	Time stream s_i holds resource r_j per access	time units	Edge annotation
$f(s_i, r_j)$	Access frequency	Rate at which stream s_i requires resource r_j	accesses/time	Edge annotation
$c(s_i, r_j)$	Criticality	Whether stream s_i blocks when r_j unavailable; 1 = blocks, 0 = can proceed	dimensionless [0,1]	Edge annotation
$\rho(r_j)$	Resource utilization	Fraction of time resource r_j is occupied; $\rho = \sum_i f(s_i, r_j) \times h(s_i, r_j)$	dimensionless [0,1+)	Derived from edge annotations
$W(r_j)$	Expected wait time	Expected time a stream waits when r_j is occupied; $W = \rho / (\mu(1 - \rho))$ for M/M/1	time units	Derived from ρ and service rate
$\kappa(s_i, s_j, r_k)$	Cascade coupling coefficient	Expected cascade promotions in s_i per unit of s_j 's activity on shared resource r_k	promotions/activity unit	Derived from dissolution inventories and validity envelopes
$\text{coord}(r_j)$	Coordination cost per access	Ops required for access protocol on resource r_j (acquire, release, verify)	ops/access	Resource property
$\text{deg}(r_j)$	Resource contention degree	Number of streams connected to resource r_j	count	Graph property
$\text{exp}(s_i)$	Stream exposure	Number of resources stream s_i requires	count	Graph property

Symbol	Name	Definition	Unit	Domain
$\text{overlap}(s_i, s_j)$	Contention overlap	Number of resources shared by streams s_i and s_j	count	Graph property
$\text{tax}(s_i, G)$	Concurrency tax	Total overhead for stream s_i in contention graph G ; sum of five components	ops + time units	Derived from G
$\delta(\text{expert}, \text{novice}, G)$	Tax discount	Ratio of expert tax to novice tax on same graph; $\delta < 1$	dimensionless (0,1)	Derived from dissolution state
n^*	Optimal stream count	Stream count maximizing total throughput; $n^* = \text{argmax } n \{n \times \text{work} - \text{tax}(n, G)\}$	count	Derived from G and scaling law

Table B: Five Tax Components

Component	Mechanism	What It Costs	Op Count Change	Op Duration Change	Pipeline State	Scales With	Primary Intervention
Contention	Stream needs occupied resource; waits	Time budget consumed waiting	None; same ops	Inflated by wait time	Occupied (waiting)	Resource utilization ρ ; nonlinear divergence as $\rho \rightarrow 1$	Resource replication

Component	Mechanism	What It Costs	Op Count Change	Op Duration Change	Pipeline State	Scales With	Primary Intervention
Cascade	Competing stream's activity invalidates dissolved processing	Ops added from dissolution promotions	Increased; promoted elements cost ops again	Unchanged per op	Active (re-processing)	Cascade coupling $\kappa \times$ competing activity rate $\times$ dissolution inventory size	Isolation; wider validity envelopes
Coordination	Managing concurrent access protocols	Ops spent on lock/handoff	Increased; coordination ops protocol added	Unchanged per op	Active (overhead work)	Number of shared resources $\times$ access frequency $\times$ protocol cost	Protocol simplification; coordination dissolution
Blocking	Pipeline idle waiting for critical resource	Dead time; zero ops executed	None; no ops during block	N/A (no op executing)	Idle (dead)	Critical resource utilization; occupancy duration by competing streams	Asynchronous decoupling

Component	Mechanism	What It Costs	Op Count Change	Op Duration Change	Pipeline State	Scales With	Primary Intervention
Interleave	Choosing which stream to service next	Ops spent on scheduling decision	Increased; decision ops added	Unchanged per op	Active (meta-decision)	Number of ready streams; decision complexity	Scheduling policy; interleave dissolution

Table C: Tax Component Equations

Component	Equation	Variables	Queueing Model Basis
Contention	$\text{contention}(s_i, G) = \sum_j f(s_i, r_j) \times W(r_j)$	f = access frequency; W = expected wait per access	M/M/1: $W = \rho / (\mu (1 - \rho))$; M/D/1: $W = \rho / (2 \mu (1 - \rho))$; M/G/1: Pollaczek-Khinchine formula
Cascade	$\text{cascade}(s_i, G) = \sum_j \neq_i \sum_k \kappa(s_i, s_j, r_k) \times \text{activity}(s_j, r_k) \times \text{recovery cost}(s_i)$	κ = coupling coefficient; activity = usage rate; recovery = ops to restabilize	Not queueing-derived; from dissolution geometry
Coordination	$\text{coordination}(s_i, G) = \sum_j f(s_i, r_j) \times \text{coord}(r_j)$	f = access frequency; coord = protocol cost per access	Direct summation; no queueing
Blocking	$\text{blocking}(s_i, G) = \sum_j f(s_i, r_j) \times c(s_i, r_j) \times W(r_j)$	f = access frequency; c = criticality; W = expected wait	Same queueing models as contention; filtered by criticality
Interleave	$\text{interleave}(s_i, G) = \text{preemption rate}(s_i, G) \times \text{interleave cost}(\text{active streams}(G))$	preemption rate from competing stream arrivals; interleave cost from decision complexity	Arrival-rate-driven; cost function domain-specific

Component	Equation	Variables	Queueing Model Basis
Total	$\text{tax}(s_i, G) = \text{contention} + \text{cascade} + \text{coordination} + \text{blocking} + \text{interleave}$	All above	Composite
In situ cost	$\text{cost insitu}(s_i) = \text{cost isolated}(s_i) + \text{tax}(s_i, G)$	Isolated cost + total tax	—
System cost	$\text{cost total}(G) = \sum_i \text{cost isolated}(s_i) + \sum_i \text{tax}(s_i, G)$	Sum across all streams	—

Table D: Architectural Motifs

Motif	Structure	Resource Degree	Contention Scaling	Cascade Scaling	Coordination Scaling	Total Tax Scaling	Examples
Star	One central resource connected to all n streams	Central: n ; periphery: 1	$O(n/(1-n\bar{f}h))$; diverges as central resource saturates	Moderate; cascade mediated only through central resource	$O(n)$ per access to central resource	Dominated by contention; steep growth, hard ceiling at saturation	Single DB connection pool; single CI pipeline; single meeting room; surgeon's visual attention

Motif	Structure	Resource Degree	Contention Scaling	Cascade Scaling	Coordination Scaling	Total Tax Scaling	Examples
Chain	Sequential pair-wise sharing; stream i shares resource with stream i+1	All resources: 2	$O(1)$ per link; but delays propagate through chain	Nearest-neighbor only; bounded coupling	$O(n)$ total across chain links	Linear with chain length; propagation adds latency not throughput loss	Pipeline architectures; assembly lines; sequential approval chains
Complete (all-to-all)	Every stream shares at least one resource with every other	Near n for shared resources	$O(n^2)$ aggregate; maximum possible contention	Maximum; every stream can disrupt every other	$O(n^2)$ pairwise; dominates at scale	Quadratic; maximum possible tax for given n	Open-plan offices; shared-everything multiprocessor; team meetings; shared mutable state

Motif	Structure	Resource Degree	Contention Scaling	Cascade Scaling	Coordination Scaling	Total Tax Scaling	Examples
Partitioned streams	in groups of size k; sharing within groups only	k per group resource	$O(k/(1-kf))$ per group; independent across groups	Within-group only; bounded by k	$O(k)$ per group	Bounded by group size; scales with k not n; groups independent	NUMA domains; pod-based teams; microservice architectures; departmental resource pools
Hierarchical resources	in tiers: local (few streams), global (many streams)	Local: 2–4; global: n	$O(\log n)$ typical (most accesses local); $O(n)$ penalty for global access	Local cascades bounded; global cascades rare but severe	$O(\log n)$ typical; $O(n)$ for global decisions	Logarithmic common case; periodic global-access penalties	Cache hierarchies (L1/L2/L3); organizational hierarchies; federated systems

Motif	Structure	Resource Degree	Contention Scaling	Cascade Scaling	Coordination Scaling	Total Tax Scaling	Examples
Ring	Each stream shares re-sources with two neighbors; no global re-source	All re-sources: 2	O(1) per link; no global bottleneck	Nearest-neighbor; limited propagation	O(n) total but distributed	Linear but distributed; no single bottleneck; propagation bounded	Token ring networks; circular assembly; round-robin team responsibilities
Hybrid star-partition	Multiple stars, each serving a partition; stars share global re-source	Star center: k; global: partition count	Within-partition: star scaling at group size; cross-partition: low-frequency global contention	Within-partition high; cross-partition low	Within-partition: O(k); cross-partition: infrequent	Two-tier: fast within partition, slow across	Replicated database with coordinator; multi-team org with shared infrastructure; multi-socket CPU

Table E: Motif Tax Scaling Derivations

Motif	Stream Count n	Contention Formula	Saturation Point	Marginal Tax of (n+1)th Stream	Optimal n*
Star	n	$W(n) = n\bar{f}\bar{h} / (\mu (1 - n\bar{f}\bar{h}))$	n sat = $1/(\bar{f}\bar{h})$	$\Delta W = \bar{f}\bar{h} / (\mu (1 - (n+1)\bar{f}\bar{h})^2)$; grows as $n \rightarrow n \text{ sat}$	n* where marginal wait cost = marginal work gained; well below n sat
Chain	n	W total = n $\times$ W link; W link = $2\bar{f}\bar{h} / (\mu (1 - 2\bar{f}\bar{h}))$	Each link saturates at $2\bar{f}\bar{h} = 1$	Adds one link of constant cost; O(1) marginal	Limited by chain propagation delay, not contention
Complete	n	W agg $\propto n^2\bar{f}\bar{h}$; each resource sees O(n) demand	n sat = $1/(\bar{f}\bar{h})$ per resource; many resources saturate together	O(n) marginal; each new stream adds contention to n existing streams	n* typically small; scales poorly
Partitioned (k per group)	g $\times$ k (g groups)	W group = $k\bar{f}\bar{h} / (\mu (1 - k\bar{f}\bar{h}))$ per group; groups independent	k sat = $1/(\bar{f}\bar{h})$ per group	Adding to a group: same as star with group size k; adding new group: O(1) marginal tax to system	Per-group: same as star with k; system: scales by adding groups

Motif	Stream Count n	Contention Formula	Saturation Point	Marginal Tax of $(n+1)$ th Stream	Optimal n^*
Hierarchical		$W_{\text{local}} = W(k_{\text{local}})$; $W_{\text{global}} = W(n)$ but accessed at rate $f_{\text{global}} \ll f_{\text{local}}$	Global saturates at $n_{\text{sat}} = 1/(f_{\text{global}} \times h_{\text{global}})$	Marginal local cost: $O(1)$; marginal global cost: $O(1/n)$ of accesses at $O(n)$ cost per access	Determined by global access fre- quency; high n^* when global access is rare

Table F: Resource Types Across Domains

Domain	Resource Type	Typical Degree	Hold Duration	Access Frequency	Criticality	Cascade Coupling κ
Computation	cache line	Exclusive per core	1–4 cycles	Millions/ms	1 (pipeline stalls)	N/A (private)
Computation	cache bank	Shared by core pair	4–12 cycles	Hundred thousands/ms	1 (stalls on miss)	Moderate (eviction disrupts partner)
Computation	cache slice	Shared by all cores	12–40 cycles	Tens of thousands/ms	0.5 (can hit memory)	Moderate (capacity pressure from all cores)
Computation	Memory bus	Shared by all cores	4–16	50–200 cycles	Thousands/ms	1 (memory access required)
Computation	Database connection	Pool shared by threads	10–1000	1–100ms	10–1000/s per thread	1 (query blocks without connection)
						Low (connections independent once acquired)

Domain	Resource	Type	Typical Degree	Hold Duration	Access Frequency	Criticality	Cascade Coupling κ
Computational I/O channel	Disk	Shared by all processes	All	1–10ms	10–100/s aggregate	0.7 (can defer non-critical I/O)	Low (I/O mostly independent)
Surgery	Primary instrument	Shared by surgeon + assistant	2	5–60s	1–10/min	0.8 (can use alternative instrument)	Low (instrument use doesn't change context)
Surgery	Surgeon's visual focus	Single re-source	All concurrent tasks	1–30s	Continuous	1 (cannot operate blind)	Very high (visual shift breaks dissolved spatial model)
Surgery	Operative field	Shared by surgeon + assistant	2	Continuous	Continuous	1 (only one set of hands at surgical site)	Moderate (other person's activity changes tissue state)
Surgery	Anesthesia monitor attention	Single re-source	All monitoring tasks	5–30s	0.5–2/min	0.9 (most monitoring is critical)	Moderate (attention shift may miss transient events)
Office	CI pipeline	Shared by all developers	Team size	5–30min	2–10/day per developer	0.3 (can work on other tasks while waiting)	Low (CI activity doesn't change developer's context)

Domain	Resource	Type	Typical Degree	Hold Duration	Access Frequency	Criticality	Cascade Coupling κ
Office	Working memory	Single re-source per developer	All concurrent cognitive tasks	Minutes to hours	Continuous for primary task	1 (cannot think without working memory)	Very high (loading new context evicts current context entirely)
Office	Team lead attention	Single re-source	Team size	5–30min per decision	2–8/day per developer	0.5 (some decisions can wait)	Low (discussing with lead doesn't change developer's task context)
Office	Shared code-base	Shared by all developers	Team size	Hours (branch lifetime)	1–5 merges/day per developer	0.4 (merge conflicts are intermittent)	Moderate (other's merge may invalidate developer's assumptions)
Office	Meeting room / calendar	Shared by all participants	Team size	30–60min	2–6/day per person	0.6 (some meetings are mandatory)	High (meetings evict working memory context)
Manufacturing	Shared tooling	Shared by adjacent stations	2–4	Seconds to minutes	10–100/hour	0.7 (may have backup tooling)	Low (tooling use doesn't change operator's context)
Manufacturing	Conveyor	Shared by all stations	Station count	Continuous	Continuous	1 (line stops without parts)	Moderate (line speed changes affect downstream timing)

Domain	Resource	Type	Typical Degree	Hold Duration	Access Frequency	Criticality	Cascade Coupling κ
ATC	Airspace sector	Shared by all aircraft	10–40	Minutes (transit time)	N/A (aircraft present continuously)	1 (cannot leave sector uncontrolled)	Moderate (each aircraft's actions change sector geometry)
ATC	Radio frequency	Shared by controller + all aircraft	10–40	5–15s per transmission	1–3/min per aircraft	1 (communications are safety-critical)	Low (transmissions are discrete, not context-changing)

Table G: Expert Tax Discount by Component

Tax Component	Novice Cost	Expert Cost	Discount δ	Dissolution Mechanism	Time to Dissolve	Dissolution Limit
Contention	W (wait time per access)	W (same wait time)	~1.0 (no discount)	Waiting is structural; not a skill	N/A	Cannot dissolve; resource physics determines wait
Cascade — CPU context switch	~800 cache misses $\times$ 200 cycles each	~200 misses $\times$ 200 cycles (smaller working set, better locality)	~0.25	Optimized memory access patterns; tighter working sets; cache-aware algorithms	Weeks to months of optimization	Bounded by minimum working set $\times$ miss penalty

Tax Component	Novice Cost	Expert Cost	Discount δ	Dissolution Mechanism	Time to Dissolve	Dissolution Limit
Cascade — cognitive interruption	15–25 min recovery per interruption	2–5 min recovery per interruption	~0.15–0.25	Wider validity envelopes on working memory contents; dissolution under varied conditions including interruptions	Months to years	Bounded by minimum context reconstruction cost
Cascade — organizational change	Days to weeks of disruption per major change	Hours to days of disruption	~0.2–0.4	Wider organizational validity envelopes; experience with change; meta-dissolution of change management	Years of organizational experience	Bounded by minimum relationship/process reconstruction
Coordination — surgical team	5 ops per instrument handoff (request, confirm, transfer, verify, acknowledge)	1 op per handoff (anticipate-extend-accept as one dissolved unit)	~0.2	Team-specific coordination dissolution; shared mental model; implicit protocols	Months of working together	Bounded by minimum physical handoff actions

Tax Component	Novice Cost	Expert Cost	Discount δ	Dissolution Mechanism	Time to Dissolve	Dissolution Limit
Coordination — software team	3–5 ops per code review cycle (request, context- load, review, com- ment, re- spond)	1–2 ops (shared conven- tions, known code- base, pre- dictable pat- terns)	~0.3– 0.5	Shared conventions dissolve; codebase familiarity reduces context-load cost; review patterns become structural	Months on shared codebase	Bounded by minimum review verification
Coordination — meeting protocol	5–10 ops per meeting (agenda review, context estab- lish- ment, discus- sion, deci- sion, docu- menta- tion)	2–3 ops (stand- ing agenda dis- solved, context shared, deci- sions rapid)	~0.3– 0.4	Meeting protocols dissolve; team context is shared and maintained structurally	Months of regular meetings	Bounded by minimum information exchange

Tax Component	Novice Cost	Expert Cost	Discount δ	Dissolution Mechanism	Time to Dissolve	Dissolution Limit
Blocking	W (wait time)	W (same wait time; may use wait produc- tively)	~0.8– 1.0 (mini- mal dis- count on block- ing itself; dis- count on lost pro- ductiv- ity during block)	Expert can switch to useful work during block without cascade; productive blocking	Months to years of multitask- ing practice	Bounded by switch cost to alternative task
Interleave — CPU sched- uler	$O(\log n)$ per decision	$O(1)$ per decision ($O(1)$ sched- ulers exist)	Variable by algo- rithm	Algorithmic: $O(1)$ scheduler is pre-computed scheduling; no per-decision computation	Engineering time (not practice)	$O(1)$ achieved; cannot go below

Tax Component	Novice Cost	Expert Cost	Discount δ	Dissolution Mechanism	Time to Dissolve	Dissolution Limit
Interleave — cognitive task switching	3–5 ops per switching decision (evaluate urgency, assess inter- ruptibility, decide, prepare to switch, switch)	0–1 ops (structural decision based on dis- solved priority pat- terns)	~0.1– 0.2	Interleave decision patterns dissolve through repetition; urgency assessment becomes structural	Months to years	Bounded by novel urgency patterns requiring conscious evaluation

Table H: Architectural Interventions as Graph Transformations

Intervention	Graph Transformation	Target Component	Expected Tax Change	Cost of Intervention	When to Apply	Example
Resource replication	Split high-degree resource node into k lower-degree nodes; redistribute edges	Contention + Blocking	$\rho_{\text{new}} = \rho_{\text{old}}/k$; $W_{\text{new}} \ll W_{\text{old}}$ (nonlinear reduction due to queueing convexity)	Capital cost of additional resource; complexity of load distribution	When single resource utilization $\rho > 0.7$ and contention is dominant tax component	Add second CI runner; add database read replicas; add second operating room
Resource upgrade	Reduce $h(s_i, r_j)$ for all edges to resource r_j (faster resource)	Contention + Blocking	$\rho_{\text{new}} = \rho_{\text{old}} \times (h_{\text{new}}/h_{\text{old}})$; proportional reduction	Cost of faster resource	When hold duration is the primary driver of utilization	Faster CI hardware; SSD replacing spinning disk; faster network link

Intervention	Graph Transformation	Target Component	Expected Tax Change	Cost of Intervention	When to Apply	Example
Stream isolation	Remove cascade coupling edges; separate context spaces	Cascade	$\text{cascade}(s_i) \rightarrow 0$ for isolated resource edges; coupling coefficient $\kappa \rightarrow 0$	Resource duplication; memory overhead; may reduce sharing benefits	When cascade is dominant tax component; when dissolved processing is frequently broken	Process address space separation; private offices; cache partitioning; dedicated staging environments

Intervention	Graph Transformation	Target Component	Expected Tax Change	Cost of Intervention	When to Apply	Example
Protocol simplification	Reduce $\text{coord}(r_j)$ on resource edges	Coordination	Proportional to coord reduction $\times$ access frequency	Engineering effort; may reduce safety guarantees	When coordination is dominant and protocol has unnecessary steps	Replace 3-phase commit with optimistic locking; replace formal reviews with pair programming; replace full hand-off protocol with implicit coordination

Intervention	Graph Transformation	Target Component	Expected Tax Change	Cost of Intervention	When to Apply	Example
Asynchronous decoupling	Change criticality $c(s_i, r_j)$ from 1 to 0 on edges; add buffer/queue	Blocking	$\text{blocking}(s_i) \rightarrow 0$ on decoupled edges; interleave and cascade may increase modestly	Engineering complexity; buffer management; eventual consistency tradeoffs	When blocking is dominant and stream can usefully proceed without the resource	Non-blocking I/O; async code review; message queues re-placing synchronous calls; batch interrupt processing

Intervention	Graph Transformation	Target Component	Expected Tax Change	Cost of Intervention	When to Apply	Example
Scheduling policy	Replace deliberative interleave decisions with structural rules	Interleave	interleave cost(k) $\rightarrow$ $O(1)$ from $O(k)$ or $O(\log k)$	Policy design effort; reduced flexibility for non-standard situations	When interleave is frequent and decision pattern is regular	Time-blocking for developers; $O(1)$ CPU scheduler; triage protocols; defined escalation paths
Stream reduction	Remove streams from the graph	All components	Proportional reduction in all components; superlinear benefit in high-contention topologies	Lost work capacity; deferred tasks	When system is past n^* (optimal stream count); tax exceeds useful work	Reduce WIP limits; limit concurrent projects; close browser tabs; reduce meeting participants

Intervention	Graph Transformation	Target Component	Expected Tax Change	Cost of Intervention	When to Apply	Example
Topology restructuring	Transform graph motif (e.g., complete $\rightarrow$ partitioned)	All components	Changes scaling law; typically reduces tax growth rate	Organizational restructuring cost	When architecture/topology is fundamentally mismatched to workload; quadratic scaling observed	Split monolith into microservices; partition team into pods; move to NUMA-aware allocation; institute departmental resource pools

Intervention	Graph Transformation	Target Component	Expected Tax Change	Cost of Intervention	When to Apply	Example
Cascade widening	Widen validity envelopes through varied training (no graph change)	Cascade	Reduces κ on existing edges without changing graph structure	Training time and cost	When cascade is dominant and processors have narrow envelopes	Cross-training; varied simulation; progressive complexity exposure; multi-environment deployment practice

Table I: Human Processor Contention Model

Stream (Concurrent Demand)	Resources Re- quired	Hold Dura- tion	Access Fre- quency	Criticality	Cascade Coupling (to primary task working memory)	Typical Tax per Instance
Primary develop- ment task	Working mem- ory (long hold), visual atten- tion (long hold), motor system (inter- mit- tent), code- base (long hold)	Hours (ses- sion)	Continuous	1.0	N/A (is the primary context)	0 (this is the isolated work)
Slack message (simple)	Visual atten- tion (brief), work- ing mem- ory (brief — no context change needed)	10– 30s	5– 15/day	0.1	Low ($\kappa \approx 0.1$; doesn't evict primary context)	30–60s (read + respond)

Stream (Concurrent Demand)	Resources Re- quired	Hold Dura- tion	Access Fre- quency	Criticality	Cascade Coupling (to primary task working memory)	Typical Tax per Instance
Slack message (complex)	Visual atten- tion (ex- tended), work- ing mem- ory (full load — differ- ent context re- quired)	2– 10min	2– 5/day	0.3	Very high ($\kappa \approx 0.9$; evicts primary task context)	20– 30min (2–5min direct + 15– 25min cascade recovery)
Code review request	Visual atten- tion (ex- tended), work- ing mem- ory (full load — differ- ent code- base con- text), code- base (differ- ent branch)	15– 45min	1– 3/day	0.4	Very high ($\kappa \approx 0.95$; complete context switch)	45– 90min (review time + recovery time)

Stream (Concurrent Demand)	Resources Required	Hold Duration	Access Frequency	Criticality	Cascade Coupling (to primary task working memory)	Typical Tax per Instance
Scheduled meeting	Visual attention, working memory (meeting context), commu- nica- tion band- width	30– 60min	2– 4/day	0.7 (most meet- ings are ex- pected)	Very high ($\kappa \approx 0.9$; meeting context displaces development context)	60– 120min (meeting + post- meeting cascade recovery)
Unscheduled interrup- tion (tap on shoulder)	Visual attention (immediate), working memory (forced partial load)	1– 5min	1– 4/day	0.5	High ($\kappa \approx 0.7$; forced attention shift disrupts working memory)	15– 30min (interrup- tion + cascade recovery)
CI pipeline wait	CI pipeline re- source (block- ing if waiting; non- blocking if async)	5– 30min	2– 6/day	0.3 (can work on other tasks)	Low ($\kappa \approx 0.1$; CI wait doesn't change cognitive context unless developer watches it)	5–30min blocking or 1–2min interleave (task- switch decision)

Stream (Concurrent Demand)	Resources Re- quired	Hold Dura- tion	Access Fre- quency	Criticality	Cascade Coupling (to primary task working memory)	Typical Tax per Instance
Production incident alert	All re- sources (imme- diate full pri- ority)	30min– hours	0.1– 1/day (varies)	1.0	Maximum ($\kappa \approx 1.0$; complete context evacuation)	1–4 hours (incident response + full cascade recovery + re-entry to primary task)
Background worry/anxiety	Working mem- ory (pervis- tent partial occupa- tion)	Hours to days	Continuous back- ground	0.2	Moderate ($\kappa \approx 0.4$; persistent low-grade disruption of working memory capacity)	10–30% contin- uous through- put reduction (working memory partially occupied)

Table J: Tax Component Dominance by Domain

Domain	Dominant Com- ponent	Second Com- ponent	Third Com- ponent	Contention (%)	Cascade (%)	Coordination (%)	Blocking (%)	Interleave (%)	Total Tax as % of Budget
Multi-threaded software (high con- tention)	Contention	Blocking	Cascade	40	20	15	20	5	30–60%

Dominant Component	Second Component	Tertiary	Contention (%)	Cascade (%)	Coordination (%)	Blocking (%)	Interleave (%)	Total Tax as % of Budget
Multi-process CPU (cache-sensitive)	Cascade	Contention	30	45	5	10	20	20–50%
Software developer (open office)	Cascade	Coordination	10	45	25	10	15	40–70%
Software developer (remote, async)	Blocking	Cascade	Coordination	20	20	40	15	20–40%
Surgical team (routine procedure)	Coordination	Contention	Cascade	15	15	45	15	15–30%
Surgical team (complex/unexpected)	Cascade	Coordination	Interleave	40	25	5	20	30–60%
Air traffic control (moderate load)	Interleave	Coordination	Contention	15	25	10	40	20–40%

	Dominant Com- ponent	Second Com- ponent	Tertiary	Contention (%)	Cascade (%)	Coordination (%)	Blocking (%)	Interleave (%)	Total Tax as % of Budget
Air traf- fic con- trol (high load)	Contention	Cascade	Interleave	35	25	15	5	20	50–80%
Manufacturing (as- sem- bly line)	Blocking	Contention	Coordination	25	5	15	45	10	15–35%
Emergency de- partment	Cascade	Interleave	Coordination	10	35	20	10	25	40–70%
Restaurant kitchen (ser- vice)	Interleave	Coordination	Contention	15	15	25	10	35	30–50%
Orchestra (per- for- mance)	Coordination	Cascade	Interleave	5	25	45	5	20	10–20%

Table K: Brooks's Law Derivation

Team Size n	Isolated Work Capacity (linear)	Pairwise Coordination Edges	Coordination Cost (quadratic)	Net Through- put	Marginal Throughput of nth Member
1	1.0	0	0	1.00	1.00
2	2.0	1	0.10	1.90	0.90
3	3.0	3	0.30	2.70	0.80
4	4.0	6	0.60	3.40	0.70
5	5.0	10	1.00	4.00	0.60
6	6.0	15	1.50	4.50	0.50
8	8.0	28	2.80	5.20	0.30
10	10.0	45	4.50	5.50	0.10

Team Size n	Isolated Work Capacity (linear)	Pairwise Coordination Edges	Coordination Cost (quadratic)	Net Through- put	Marginal Throughput of nth Member
11	11.0	55	5.50	5.50	0.00 (crossover)
12	12.0	66	6.60	5.40	-0.10
15	15.0	105	10.50	4.50	-0.40
20	20.0	190	19.00	1.00	-0.80

Assumptions: Each person contributes 1.0 unit of isolated work. Each coordination edge costs 0.10 units. Edges = $n(n-1)/2$. Coordination cost = edges $\times$ cost per edge. Net throughput = isolated – coordination. Crossover at $n^* \approx 11$ for these parameters.

General formula: $n^* = (1 + \sqrt{1 + 2/c_edge}) / 1$, where c_edge is the coordination cost per pairwise edge. Lower c_edge (more efficient coordination) raises n . *Higher c_edge (expensive coordination) lowers n .*

Note: This model uses only the coordination component. Including contention (shared codebase, CI pipeline) and cascade (interruptions between team members) lowers n^* further. The table represents a lower bound on tax.

Table L: Computational Measurement Specifications

Tax Component	CPU Measure-ment	Hardware Counter / Tool	Baseline (Isolated)	In Situ Measurement	Tax = In Situ – Isolated
Contention (memory bus)	Memory bandwidth saturation delay	IMC (Integrated Memory Controller) counters; bandwidth utilization	Single-process memory bandwidth	Multi-process memory bandwidth; per-process latency inflation	Additional latency per access $\times$ access count
Contention (cache)	Cache line contention	HITM (Hit Modified) counters; false sharing detection	Single-process cache hit rate	Multi-process HITM rate; invalidation overhead	HITM events $\times$ invalidation penalty
Cascade (L1)	Cache misses after context switch	L1-dcache-load-misses; context switch counter	Steady-state miss rate (no switches)	Miss rate in first N accesses after each context switch	Additional misses $\times$ (miss penalty – hit cost)

Tax Component	CPU Measurement	Hardware Counter / Tool	Baseline (Isolated)	In Situ Measurement	Tax = In Situ – Isolated
Cascade (L2/L3)	Cache pollution from competing processes	LLC-load-misses; perf per-process	Single-process LLC miss rate	Multi-process LLC miss rate	Additional LLC misses $\times$ memory latency
Cascade (branch predictor)	Misprediction after workload change	branch-misses counter	Steady-state misprediction rate	Misprediction spike after context switch	Additional mispredictions $\times$ pipeline flush cost
Cascade (TLB)	TLB misses after context switch	dTLB-load-misses	Steady-state TLB miss rate	Miss rate after context switch (without PCID) vs with PCID	Additional TLB misses $\times$ page walk cost
Coordination (locks)	lock acquisition overhead	lock contention profilers (perf lock, mutrace)	Uncontended lock cost	Contended lock cost	Wait time + spin time per contended acquisition
Coordination (atomics)	Atomic operation overhead under contention	Atomic operation latency profiling	Uncontended atomic cost (~few cycles)	Contended atomic cost (potentially hundreds of cycles)	Contended – uncontended per atomic op
Blocking (I/O)	I/O wait time	iowait in /proc/stat; per-process I/O wait	I/O latency with dedicated device	I/O latency under concurrent access	Additional I/O wait from queue depth
Blocking (scheduler)	Involuntary context switches	/proc/[pid]/status nonvoluntary ctxt switches	0 (single process, no preemption)	Involuntary switch count $\times$ (switch cost + cascade cost)	Total involuntary switch overhead

Tax Component	CPU Measurement	Hardware Counter / Tool	Baseline (Isolated)	In Situ Measurement	Tax = In Situ – Isolated
Interleave (scheduler)	Scheduler decision time	sched debug; ftrace scheduler tracing	N/A (single process, no decision)	Time in scheduler per decision $\times$ decisions per second	Total scheduler overhead per process

Table M: Prediction Testing Specifications

Prediction	Independent Variable	Dependent Variable	Measurement Method	Control Requirements	Expected Result	Falsification Criteria
1. Tax predictability	Contention graph annotation completeness	Prediction error: predicted tax – measured tax	Build graph from profiling; predict tax; measure actual; compare	Stable workload during measurement period; all significant resources identified	Prediction error decreases monotonically with annotation completeness; $<20\%$ error with full annotation	Error does not correlate with annotation completeness; or $>50\%$ error with full annotation
2. Intervention specificity	Type of intervention (replication, isolation, etc.)	Change in each tax component	Measure all five components before and after intervention	Single intervention per measurement; stable workload; sufficient measurement duration	Targeted component decreases; non-targeted components unchanged (within noise)	Non-targeted components change by $>20\%$ of targeted component's change

Prediction	Independent Variable	Dependent Variable	Measurement Method	Control Requirements	Expected Result	Falsification Criteria
3. Motif-determining scaling	Stream count n ; graph topology held constant	Total tax as function of n	Add streams incrementally; measure total tax at each n	Topology preserved; per-stream workload constant; resources scaled with topology	Star/complete: superlinear. Chain: linear. Partitioned: constant per group. Hierarchical: logarithmic	Observed scaling law doesn't match motif prediction for given topology
4. Expert discount components	Processor experience level (expert vs novice)	Each tax component separately	Measure all five components for expert and novice on same graph	Same contention graph; same workload; same resources; experience as only variable	δ cascade and δ interleave $\ll 1$; δ contention and δ blocking ≈ 1	Expert shows no discount on cascade/interleave; or shows large discount on contention/blocking
5. Brooks's Law crossover	Team size	Net throughput (isolated work – total tax)	Measure team throughput at different sizes; control for personnel quality	Same project type; same tooling; team members of comparable individual capability	Throughput peaks at n^* then declines; n^* computable from graph	Throughput increases linearly with n indefinitely; or peaks at n unrelated to graph prediction

Prediction	Independent Variable	Dependent Variable	Measurement Method	Control Requirements	Expected Result	Falsification Criteria
6. Optimal stream count	Stream count n for fixed graph	Total system throughput	Run system at varying stream counts; measure aggregate completed work per time unit	Fixed resources; fixed per-stream workload; sufficient run duration at each n	Peak at predicted n*; decline beyond	Peak at n significantly different from predicted n*; or no peak observed

Table N: Cross-Level Contention Mapping

Level	Processor	Pipeline	Streams	Shared Resources	Time Budget	Characteristic Tax	Dominant Component
Transistor	Logic gate	Signal propagation	Concurrent signal paths	Shared buses, fan-out nodes	Clock cycle	Wire delay, capacitive loading	Contention (electrical)
Instruction	CPU core	Instruction pipeline	Pipeline stages; out-of-order execution windows	Execution units, register file, load/store buffer	Pipeline depth in cycles	Pipeline stalls, structural hazards	Contention + Cascade (branch mispredict)
Process	CPU + cache	Scheduling quantum	Concurrent processes	Cache hierarchy, memory bus, I/O	Process time slice	Context switch overhead	Cascade (cache) + Interleave (scheduler)

Level	Processor	Pipeline	Streams	Shared Resources	Time Budget	Characteristic Tax	Dominant Component
Application	Developer	Conscious attention	Concurrent tasks within one work session	Working memory, visual attention, motor	Work session (hours)	Interruption recovery	Cascade (working memory)
Team	Team lead / manager	Decision band-width	Concurrent team members' work	Codebase, CI, review pool, lead attention	Sprint/iteration	Coordination overhead	Coordination + Blocking
Organization	Executive leadership	Strategic band-width	Concurrent departments/projects	Budget, headcount, infrastructure, brand	Quarter/year	Alignment overhead	Coordination + Interleave
Market	Industry participants	Market attention	Competing firms	Customer attention, talent pool, capital	Market cycle	Competitive interference	Contention (market share)

Table O: Specification Summary

Metric	Count
Formal definitions	16
Tax components defined	5 (contention, cascade, coordination, blocking, interleave)
Tax component equations	5 + 2 composite (per-stream total, system total)
Architectural motifs characterized	7 (star, chain, complete, partitioned, hierarchical, ring, hybrid)
Motif scaling derivations	5 (star, chain, complete, partitioned, hierarchical)
Resource types catalogued	28 (across 6 domains)
Expert discount components	5 (one per tax component)
Architectural interventions	9
Human processor stream types	9
Domain tax profiles	12

Metric	Count
Computational measurement specifications	11
Cross-level contention mappings	7 levels
Testable predictions	6
Open problems	5

Key Equations Summary

Resource utilization:

$$\rho(r_j) = \sum_i f(s_i, r_j) \times h(s_i, r_j)$$

Expected wait time (M/M/1):

$$W(r_j) = \rho(r_j) / (\mu_j \times (1 - \rho(r_j)))$$

Contention cost:

$$\text{contention}(s_i, G) = \sum_j f(s_i, r_j) \times W(r_j)$$

Cascade cost:

$$\text{cascade}(s_i, G) = \sum_j \neq_i \sum_k \kappa(s_i, s_j, r_k) \times \text{activity}(s_j, r_k) \times \text{recovery_cost}(s_i)$$

Coordination cost:

$$\text{coordination}(s_i, G) = \sum_j f(s_i, r_j) \times \text{coord}(r_j)$$

Blocking cost:

$$\text{blocking}(s_i, G) = \sum_j f(s_i, r_j) \times c(s_i, r_j) \times W(r_j)$$

Interleave cost:

$$\text{interleave}(s_i, G) = \text{preemption_rate}(s_i, G) \times \text{interleave_cost}(\text{active_streams}(G))$$

Total concurrency tax:

$$\text{tax}(s_i, G) = \text{contention} + \text{cascade} + \text{coordination} + \text{blocking} + \text{interleave}$$

In situ cost:

$$\text{cost_insitu}(s_i) = \text{cost_isolated}(s_i) + \text{tax}(s_i, G)$$

Tax discount:

$$\delta(\text{expert}, \text{novice}, G) = \text{tax}(\text{expert}, G) / \text{tax}(\text{novice}, G)$$

Optimal stream count:

$$n^* = \text{argmax}_n \{ n \times \text{work_isolated} - \text{tax_total}(n, G) \}$$

Brooks's Law crossover (complete graph, coordination only):

$n^* = (1 + \sqrt{1 + 2/c_edge})$ where c_edge = coordination cost per pairwise edge

Fundamental inequality (from prior work, referenced throughout):

$$\Sigma ops \times \bar{d} \leq N$$

[[HOWL-MATH-18-2026](#)] Concurrency Tax from System Structure. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-18-2026>

[[HOWL-INFO-11-2026](#)] The Relationship of Zero, One, and Infinity in Information Processing. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-11-2026>

[[HOWL-INFO-12-2026](#)] Information Processing Requires Reduction to Cardinality One. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-12-2026>

[[HOWL-INFO-13-2026](#)] The Six States of Information. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO-13-2026>

[[HOWL-MATH-14-2026](#)] A Mathematical Theory of Processing. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-14-2026>

[[HOWL-MATH-15-2026](#)] A Measurement Theory of Processing. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-15-2026>

[[HOWL-MATH-16-2026](#)] The Geometry of Dissolution and Fragility. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-16-2026>

[[HOWL-MATH-17-2026](#)] Processing Entropy as a Metric Space. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-17-2026>